

Laboratório 3 - Representação de Informação

 ic.unicamp.br/~allanms/mc404-S22026/labs/Lab-03

Ponto Flutuante no RISC-V (Extensão F)

A arquitetura base RISC-V de 32 bits (RV32I) nativamente contém apenas operações com valores inteiros. Para realizar operações matemáticas com números reais (valores fracionários) diretamente em hardware, o processador precisa implementar a **Extensão F (Single-Precision Floating-Point)**.

A Extensão F adiciona 32 registradores de ponto flutuante (**f0** a **f31**) que adotam o padrão internacional **IEEE 754 de Precisão Simples** para representar valores fracionários.

Entendendo e Decodificando o Padrão IEEE 754 (32 bits)

O padrão IEEE 754 divide os 32 bits de um número em 3 partes: **Sinal**, **Expoente** e **Mantissa (ou Fração)**.

- **Bit 31 (1 bit):** Sinal (S)
- **Bits 30 a 23 (8 bits):** Expoente (E)
- **Bits 22 a 0 (23 bits):** Mantissa (M)

A fórmula matemática que define o valor do número decimal a partir desses bits é:

$$Valor = (-1)^S \times (1 + M) \times 2^{(E - 127)}$$

Decodificação com Truncmento

Para entender na prática, vamos decodificar o binário **11000001011100000000000000000000**, extraíndo apenas a parte inteira do valor.

1. Sinal (S): Primeiro bit (bit 31).

- Se for **0**, o número é positivo. Se for **1**, é negativo.
- No exemplo: **1** (negativo).

2. Expoente (E): Próximos 8 bits.

- Calculamos o valor decimal desses 8 bits e subtraímos o "viés" (bias) que no caso do IEEE 754 32-bits é sempre 127.
- No exemplo: **1000010**. Convertendo para decimal: $(128 + 2) - 127 = 130 - 127 = 3$.

3. Mantissa (M): Últimos 23 bits.

- #### 4. Resultado Final: -15

Escreva um programa em **C** que lê uma string coposta por 0's e 1's da entrada padrão, interprete-a como sendo os bits de um número de ponto flutuante (float) de 32 bits no padrão IEEE 754, e imprima as representações da parte inteira especificadas a continuação.

Código Base

```
#define STDIN_FD 0
#define STDOUT_FD 1

int read(int __fd, const void *__buf, int __n){
    int ret_val;
    __asm__ __volatile__(
        "mv a0, %1      # file descriptor\n"
        "mv a1, %2      # buffer \n"
        "mv a2, %3      # size \n"
        "li a7, 63      # syscall read code (63) \n"
        "ecall          # invoke syscall \n"
        "mv %0, a0      # move return value to ret_val\n"
        : "=r"(ret_val) // Output list
        : "r"(__fd), "r"(__buf), "r"(__n) // Input list
        : "a0", "a1", "a2", "a7"
    );
    return ret_val;
}

void write(int __fd, const void *__buf, int __n)
{
    __asm__ __volatile__(
        "mv a0, %0      # file descriptor\n"
        "mv a1, %1      # buffer \n"
        "mv a2, %2      # size \n"
        "li a7, 64      # syscall write (64) \n"
        "ecall"
        : // Output list
        : "r"(__fd), "r"(__buf), "r"(__n) // Input list
        : "a0", "a1", "a2", "a7"
    );
}

void exit(int code)
{
    __asm__ __volatile__(
        "mv a0, %0      # return code\n"
        "li a7, 93      # syscall exit (64) \n"
        "ecall"
        : // Output list
        : "r"(code) // Input list
        : "a0", "a7"
    );
}

int main()
{
```

```
/* your logic */

/* IEEE 754 -> integer */
/* integer -> decimal */
/* integer -> binary */
/* binary -> hexadecimal */

return 0;
}

void _start()
{
    int ret_code = main();
    exit(ret_code);
}
```

Entrada do Programa

Uma string com 32 caracteres (composta apenas por 0's e 1's), seguida por um caractere de nova linha ("**\n**").

Saída do Programa

Após a leitura, e interpretada a entrada como um valor representado no padrão IEEE 754, o programa deve extrair a parte inteira (truncando a parte fracionária) e imprimir as seguintes representações:

- a **parte inteira** do número, representado na **base decimal**, é precedida pelo **sinal se o mesmo for negativo**;
- a representação **binária** (complemento de 2) após a conversão para inteiro, precedida por "**0b**" em **Big-endian**;
- a representação **binária** (complemento de 2) após a conversão para inteiro, precedida por "**0b**" em **Little-endian**;
- a representação **hexadecimal** (complemento de 2) após a conversão para inteiro, precedida por "**0x**" em **Big-endian**;
- a representação **hexadecimal** (complemento de 2) após a conversão para inteiro, precedida por "**0x**" em **Little-endian**.

Exemplos

```
| 0xf1ffffff
```

14/09/2026, 11:10